Advanced C Course

Code Quality applied to C - Session 4

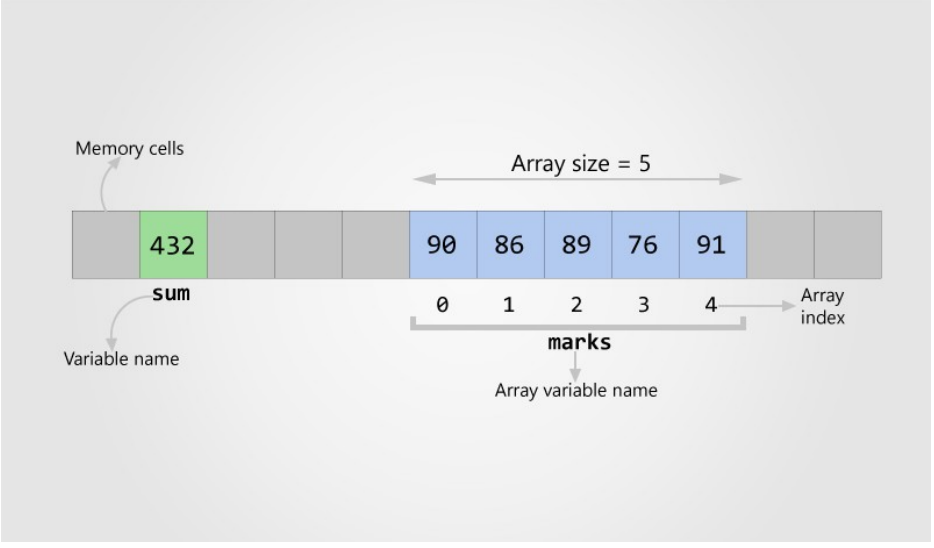
Pascal Rothnemer

23 February 2026, EPITA

Arrays

Data structure holding **finite sequential** collection of **homogeneous** data.

- **finite** – nb of data is finite, fixed prior to use.
- **sequential** in memory → can be indexed.
- **homogeneous** – The data are of the same type.
- **One-dimensional** or **Multi-dimensional** array



Arrays

```
int marks[5]; // defines a one-dimensional array of 5 integers

for (int i =0 ; i<5 ; i++)

{

marks[i] = 0 ;// init array via a loop over array index

}
```

Arrays

```
int marks[5]; // defines a one-dimensional array of 5 integers
for (int i =0 ; i<=5 ; i++)
{
marks[i] = 0 ;// init array via a loop over array index
}
```

Arrays

```
int marks[5]; // this defines an array of 5 integers

// the line below defines and initializes an array of 5 integers

int marks[5] = { 90, 86, 89, 76, 91 };

// the lines below initialize the array element by element

marks[0] = 90; // Assigns 90 to first element of marks array

marks[1] = 86; // Assigns 86 to second element of marks array

marks[2] = 89; // Assigns 86 to second element of marks array

marks[3] = 76; // Assigns 86 to second element of marks array

marks[4] = 91; // Assigns 91 to fifth element of marks array
```

Rehearsal - Pointer

```c
#include <stdio.h>
int main()
{
   int num1 =10, num2 = 20, sum;
   int* ptr1, * ptr2;
   ptr1 = &num1;
   ptr2 = &num2;
   sum = *ptr1 + *ptr2;
   printf("Sum = %d", sum);
   return 0;
}
```

What is the output ?

Rehearsal - Pointer

```c
#include <stdio.h>
int main()
{
   int num1 =10, num2 = 20, sum;
   int* ptr1, * ptr2;
   ptr1 = &num1;
   ptr2 = &num2;
   sum = *ptr1 + *ptr2;
   printf("Sum = %d", sum);
   return 0;
}
```

30

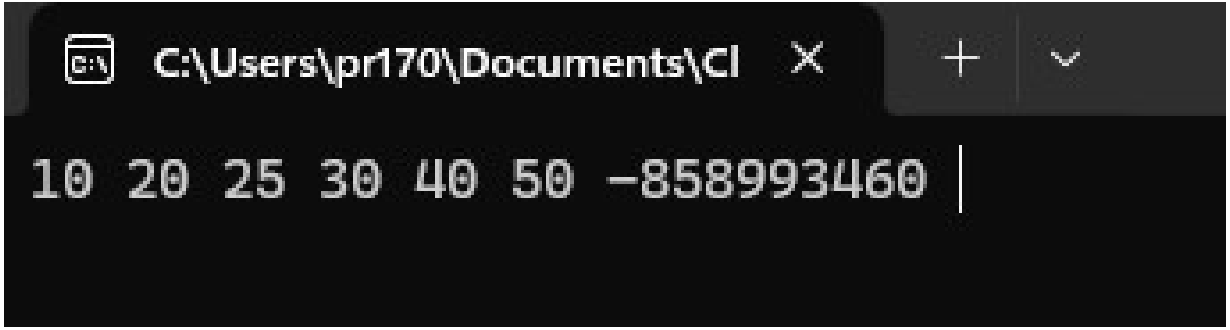
Rehearsal - Array

```
int main()
{
  int arr[6] = { 10, 20, 30, 40, 50 };
  int size = sizeof(arr);
  int length = sizeof(arr) / sizeof(arr[0]);
  int pos = 2, num = 25;
  for (int i = length-1; i > pos; i--){
    arr[i] = arr[i - 1];
    }
  arr[pos] = num;
  length++;
  for (int i = 0; i < length; i++){
    printf("%d ", arr[i]);}
  return 0;
}
```

What is the output ?

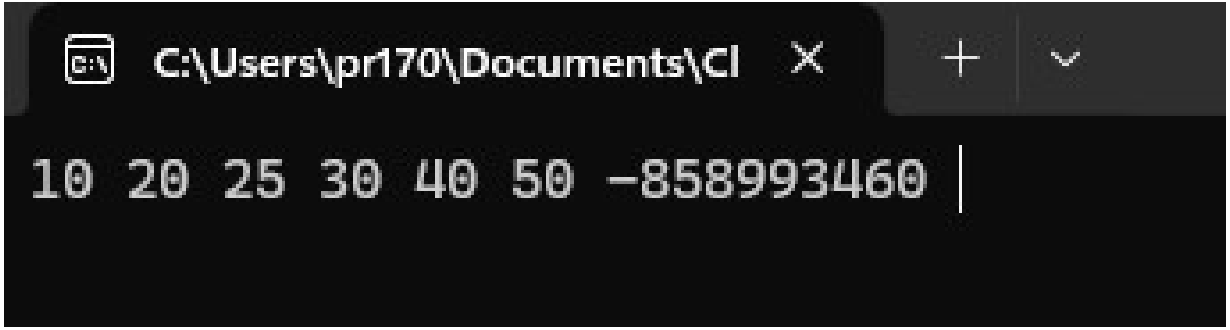
Rehearsal - Array

```
int main()
{
  int arr[6] = { 10, 20, 30, 40, 50 };
  int size = sizeof(arr);
  int length = sizeof(arr) / sizeof(arr[0]);
  int pos = 2, num = 25;
  for (int i = length-1; i > pos; i--){
    arr[i] = arr[i - 1];
    }
  arr[pos] = num;
  length++;
  for (int i = 0; i < length; i++){
    printf("%d ", arr[i]);}
  return 0;
}
```



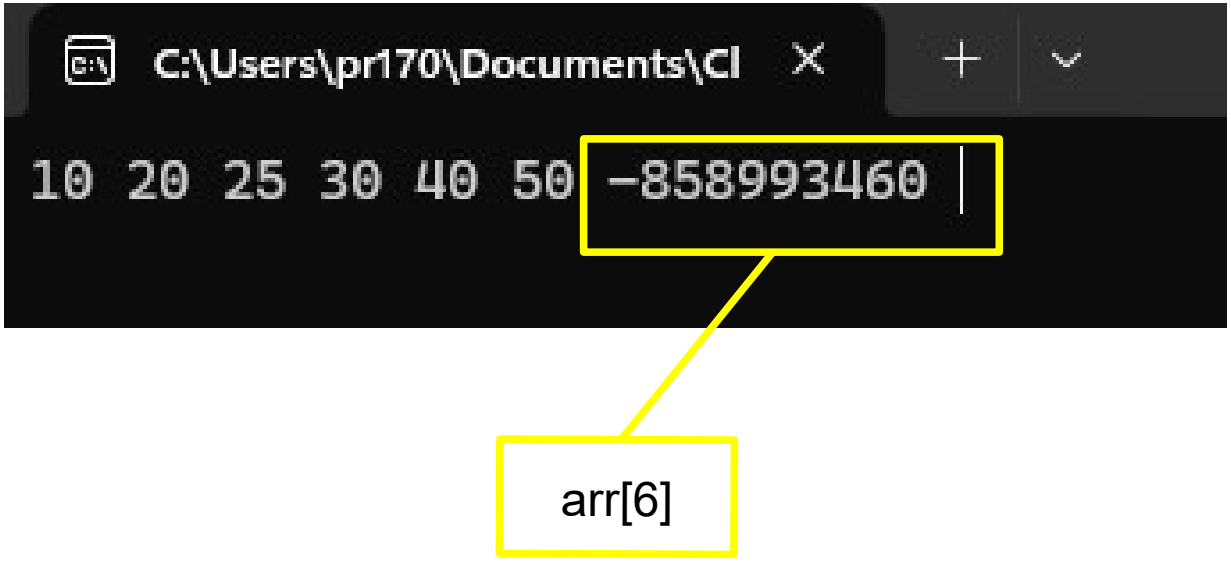
Rehearsal - Array

```c
int main()
{
  int arr[6] = { 10, 20, 30, 40, 50 };
  int size = sizeof(arr);
  int length = sizeof(arr) / sizeof(arr[0]);
  int pos = 2, num = 25;
  for (int i = length-1; i > pos; i--){
    arr[i] = arr[i - 1];
    }
  arr[pos] = num;
  length++;
  for (int i = 0; i < length; i++){
    printf("%d ", arr[i]);}
  return 0;
}
```



Rehearsal - Array

```
int main()
{
  int arr[6] = { 10, 20, 30, 40, 50 };
  int size = sizeof(arr);
  int length = sizeof(arr) / sizeof(arr[0]);
  int pos = 2, num = 25;
  for (int i = length-1; i > pos; i--){
    arr[i] = arr[i - 1];
    }
  arr[pos] = num;
  length++;
  for (int i = 0; i < length; i++){
    printf("%d ", arr[i]);}
  return 0;
}
```



Exercise

What is the output ?

```
int main(){
    int myNumbers[4] = { 25, 50, 75, 100 };
    // Get the value of the second element in myNumbers
    printf("%d\n", *(myNumbers + 1));
    // Get the value of the third element in myNumbers
    printf("%d", *(myNumbers + 2));
}
```

Exercise

What is the output ?

```
int main(){
    int myNumbers[4] = { 25, 50, 75, 100 };
    // Get the value of the second element in myNumbers
    printf("%d\n", *(myNumbers + 1));
    // Get the value of the third element in myNumbers
    printf("%d", *(myNumbers + 2));
}
```

```
50
75
```

Arrays – exercise

write a Program that

- defines an array of 11 integers
- Let user enter 11 integer values
- Calculate average, display it
- Calculate median, display it

TIP : sort array - the (11+1)/2 = 6th element of sorted array is the median of the original array

Arrays – sorting by ascending order

```
int i, j, c;
for (i = 0; i < N; i++)
    for (j = i + 1; j < N; j++)
        if (T[i] > T[j]) {
            c = T[i];
            T[i] = T[j];
            T[j] = c;
        }
```

- Find the minimum of the array and save it in 1st position.
- Once done do the same for the remaining part of the array by positioning the new minimum at the second location, and so forth…
- At the end of this double loop, the array is sorted by ascending order

Arrays – Dynamic Memory Allocation

```
int marks[10]; // array length is 10. What if we need to change it
```

A runtime case needs only 6 elements → 4 indices are wasting memory. We need to decrease length from 10 to 5.

If on contrary there is a need for 3 more elements, 3 indices more are required. The length needs to increase from 10 to 13.

This is possible thanks to **Dynamic Memory Allocation** in C : the size of an Array can be changed at runtime. 4 functions provided by C under <stdlib.h> are :

Malloc(), calloc(), free(), realloc()

Arrays – Dynamic Memory Allocation

```
ptr1 = (int*) malloc(100 * sizeof(int)); //allocates 100 int

ptr2 = (float*) calloc(25, sizeof(float));//contiguous alloc

[...]

free(ptr1);// de-allocates the memory.

free(ptr2);// de-allocates the memory.

[...]

ptr1 = realloc(ptr, newSize);// changes memory allocation
```

Arrays – Dynamic Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int* ptr = (int*)malloc(20);// allocates the memory.
    // Populate & print the array
    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;
        printf("%d ", ptr[i]);
   free(ptr1);// de-allocates the memory.
return 0;
}
```

Multidimensional Arrays

```
float x[3][4]; // 2D array.
// holds 3x4 = 12 elements.
// think/see it as a 3 rows x 4
// columns matrix.
```

	Column 1	Column 2	Column 3	Column 4
Row 1	x[0][0]	x[0][1]	x[0][2]	x[0][3]
Row 2	x[1][0]	x[1][1]	x[1][2]	x[1][3]
Row 3	x[2][0]	x[2][1]	x[2][2]	x[2][3]

Exercise

Write a small program that

- takes an input value from the user
- stores it into a local variable and
- display this same value by pointer.

Exercise

```
#include <iostream>
using namespace std;
int main() {
    int a;
    cout << "Enter number:";
    cin >> a;
    int* b = &a;
    cout << " The pointer values is " << *b << " the memory address is: " << b;
    return 0;
}
```

Exercise

Write a small program that

- takes input array daily temperatures as floats.
- findout the maximum and minimum values.

Exercise

```
double maximum(double arr[], int len) {
double max = -100;
for (int i = 0; i < len; i++) {
if (max < arr[i])
max = arr[i];
}
return max;
}
double minimun(double arr1[], int len1) {
double min = 100;
for (int u = 0; u < len1; u++) {
if (min > arr1[u])
min = arr1[u];
}
return min;
}
```

```
int main() {
int length;
cout << "How many temperatures would you like to add (MAX 100) ?\n";
cin >> length;
double array[100];
cout << "\n\n Enter the temperatures:\n";
for (int p = 0; p < length; p++)
    cin >> array[p];
double maxT, minT;
maxT = maximum(array, length);
minT = minimun(array, length);
cout << "\n\n The max. temperature of the day is: " << maxT;
cout << "\n The min. temperature of the day is :" << minT;
return 0;
}
```

C/C++ Standard Libraries

- Libraries that come automatically with Std C and Std C++
- Developed and continuously improved/enriched over time
- C++ Std library includes most of C Std library

C Standard RunTime Library

- the standard library for the C programming language, as specified in the ISO C standard.
- provides macros, type definitions and functions for tasks such as string manipulation, mathematical computation, input/output processing, memory management, and input/output.
- Its application programming interface (API) is declared in a number of header files. Each header file contains one or more function declarations, data type definitions, and macros.

C Standard RunTime Library

<assert.h>	Declares the assert macro, used to assist with detecting logical errors and other types of bugs while debugging a program.
<math.h>	Defines common mathematical functions, such as : abs, fabs, log, sqrt, pow, sin, cos, exp...
<stdio.h>	Defines core input and output functions
<stdlib.h>	Defines numeric conversion functions, pseudo-random numbers generation functions, memory allocation, process control functions
<string.h>	Defines string-handling functions such as strcpy (string copy), strcmp (string compare)...
<complex.h>	Defines a set of functions for manipulating complex numbers.
...and many more, see reference	

<Algorithm>

Algorithm Reference

collection of functions designed to be used on ranges of elements.

- Non-modifying operations:
- Modifying operations:
- Partitions:
- Sorting:
- Binary search:
- Merge
- Min/max:

<Algorithm> - Sorting

Sort	Sort elements in range (function template)
stable_sort	Sort elements preserving order of equivalents (function template)
partial_sort	Partially sort elements in range (function template)
partial_sort_copy	Copy and partially sort range (function template)
is_sorted	Check whether range is sorted (function template)
is_sorted_until	Find first unsorted element in range (function template)
nth_element	Sort element in range (function template)

<Algorithm> - Min Max

- min Return the smallest
- max Return the largest
- minmax Return smallest and largest elements
- min_element Return smallest element in range
- max_element Return largest element in range
- minmax_element Return smallest and largest elts in range

<Algorithm> exercise

Write a C++ program to check whether a given number is a power of two or not.

< Algorithm> exercise solution

```
string IsPowersOfTwo(int n) {

// Loop through integers from 0 to max integer value

for (int x = 0; x < INT_MAX; x++)

// Check if 2 raised to power of 'x' is equal to 'n'

if (pow(2, x) == n)

   return "True"; // if 'n' is a power of 2}

else if (pow(2, x) > n)

   break;// If 2^^x  > 'n', break the loop

return "False"; // if 'n' not a power of 2

}
```

```
#include <iostream> // Include i/o stream library

#include <cmath> // Include math functions

using namespace std; // Using standard namespace

int main() {
// Test cases
cout << "Is 8 a power of 2: " << IsPowersOfTwo(8) << endl;
cout << "Is 256 a power of 2: " << IsPowersOfTwo(256) << endl;
cout << "Is 124 a power of 2: " << IsPowersOfTwo(124) << endl;
return 0; // Return 0 to indicate successful completion
}
```

File handling

C supports creating, reading, writing and appending data to a file, using FILE structure holding a logical reference to the actual physical file on disk.

```
#define DATA_SIZE 1000

char data[DATA_SIZE];// array to store user input */

FILE* fPtr;

fPtr = fopen("data/file1.txt", "w");

if (fPtr == NULL)

{   /* File not created hence exit */

   printf("Unable to create file.\n");

   exit(EXIT_FAILURE);

}
```

File handling – Exercise

Write a C program to

- Declare a pointer to a FILE type to store reference of file, say FILE * fPtr = NULL;.
- Create a file « myfile1.txt » using fopen() in write mode
- Get a phrase from the user, no more than 1000 characters
- Stores it into a local array
- write it into file1.txt,
- Close file1.txt
- Say everything is OK and exit

File handling

```
/* Input contents from user to store in file */

printf("Enter contents to store in file : \n");

fgets(data, DATA_SIZE, stdin);

/* Write data to file */

fputs(data, fPtr);

/* Close file to save file data */

fclose(fPtr);

/* Success message */

printf("File created and saved successfully. :) \n");
```

Standard Template Library

- The Standard Template Library (STL) is a software library originally designed by Alexander Stepanov for the C++ programming language that influenced many parts of the C++ Standard Library. It provides four components called algorithms, containers, functions, and iterators.

C++ Standard RunTime Library

- a collection of classes and functions written in the core language and part of the C++ ISO Standard itself.
- incorporates most headers of the ISO C standard library
- based upon conventions introduced by the Standard Template Library (STL)
- Not only specifies the syntax and semantics of generic algorithms, but also places requirements on their performance

C++ Standard RunTime includes C Library

- C++ incorporates the C standard library into its own.
- C functions made available in namespace « std » (e.g., C printf as C++ std::printf)
- « using namespace std » above or within main avoids to apply the std:: prefix
- <cstdio> substitutes for <stdio.h>, <cmath> for <math.h>
- note lack of .h extension on C++ header names.

C++ Standard RunTime Library

<iostream>	Provides standard input/output streams and functions.
<cstdio>	Provides functions for performing input/output operations on files.
<cstring>	Provides functions for manipulating strings and arrays of characters
<algorithm>	Provides a collection of useful algorithms such as sorting, searching...
<cmath>	Provides functions for performing mathematical operations.
<string>	Provides the string class for manipulating strings.
<vector>	Provides the vector container class for dynamic arrays
<ctime>	Provides functions for working with dates and times.
<cstdlib>	Provides general-purpose functions for memory management, random number generation...

Vectors

- part of the C++ Standard Template Library.
- A vector is like a **resizable array.**
- A **pointer** to a **dynamically allocated array**, and data members holding the **capacity** and **size** of the vector.
  - **Size** = actual number of elements
  - **Capacity** = size of the internal array.
- Unlike arrays, **the size of a vector can grow dynamically.**

We can change the size of the vector by code.
It replaces manual coding of dynamic memory allocation
To use vectors, we need to include the vector header file in our program.

Vectors

```
#include <vector>
std::vector<int> V = { 8, 4, 5, 6, 9, 11};
```

OR

```
using namespace std;
#include <vector>
vector<int> V = { 8, 4, 5, 6, 9, 11};
```

Vectors exercise

Create a console application project doing the following :

- Create an empty vector of type <string>.
- Append items, "eggs," "milk," "sugar," "chocolate,", "flour" to the vector. Print it. (hint : use « push_back »)
- Remove the last element from the vector. Print it.(hint : use « pop_back »)
- Append item "coffee" to the vector. Print it.

Vectors exercise Solution

```
vector <string> ingredients; // Create an empty
vector of type <string>
// Append "eggs," "milk," "sugar," "chocolate,"
and "flour" elements to the vector.
ingredients.push_back("eggs");
ingredients.push_back("milk");
ingredients.push_back("sugar");
ingredients.push_back("chocolate");
ingredients.push_back("flour");
//Print it.
for (std::string ingredient : ingredients) {
    std::cout << ingredient << "\n";}
```

```
// Remove the last element from the vector
ingredients.pop_back();
//Print it.
for (std::string ingredient : ingredients) {
    std::cout << ingredient << "\n";}
// Append the item, "coffee" to the vector
ingredients.push_back("coffee");
// Print it.
for (std::string ingredient : ingredients) {
    std::cout << ingredient << "\n";}
```

Overloaded Functions – C++ Only

In C++, two different functions can have the same name while their parameters are different :

- different number of parameters
- parameters of a different type

Overloaded Functions

```
int operate(int a, int b){
return (a * b);
}

float operate(float a, float b){
return (a / b);
}
```

```
int main(){
int x = 5, y = 2, ires;
float n = 5.0, m = 2.0. , fres;
ires = operate(x, y);
fres = operate(n, m);
return 0;
}
```

Functions templates – C++ only

```
int operate(int a, int b){
return (a * b);
}

float operate(float a, float b){
return (a * b);
}
```

```
// function template
#include <iostream>
using namespace std;

template <class T>
T sum (T a, T b)
{
  T result;
  result = a + b;
  return result;
}

int main () {
  int i=5, j=6, k;
  double f=2.0, g=0.5, h;
  k=sum<int>(i,j);
  h=sum<double>(f,g);
  cout << k << '\n';
  cout << h << '\n';
  return 0;
}
```

Rehearsal - Vector

```
#include <vector>
using namespace std;
int main() {
   vector<int> numbers{ 1, 2, 3, 4, 5 };
   num.push_back(6);
   num.push_back(7);
   num.pop_back();
   num.push_back(8);
   cout << "\n Vector numbers : ";
   for (const int& i : numbers) {
      cout << i << "  ";
   }
   return 0;
}
```

What is the output ?

Rehearsal - Vector

```
#include <vector>
using namespace std;
int main() {
   vector<int> numbers{ 1, 2, 3, 4, 5 };
   num.push_back(6);
   num.push_back(7);
   num.pop_back();
   num.push_back(8);
   cout << "\n Vector numbers : ";
   for (const int& i : numbers) {
      cout << i << "  ";
   }
   return 0;
}
```

1 2 3 4 5 6 8

Rehearsal - Matrix

```c
#include <stdio.h>
#define SIZE 3 // Size of the matrix
int main()
{
   int A[SIZE][SIZE] = {
      {1, 2, 3},
      {4, 5, 6},
      {7, 8, 9}
   };
   int row, sum = 0;
   for (row = 0; row < SIZE; row++)
   {
      sum = sum + A[row][row];
   }
   printf("\nSum = %d", sum);
   return 0;
}
```

What's the output ?

Rehearsal - Matrix

```c
#include <stdio.h>
#define SIZE 3 // Size of the matrix
int main()
{
   int A[SIZE][SIZE] = {
      {1, 2, 3},
      {4, 5, 6},
      {7, 8, 9}
   };
   int row, sum = 0;
   for (row = 0; row < SIZE; row++)
   {
      sum = sum + A[row][row];
   }
   printf("\nSum = %d", sum);
   return 0;
}
```

15

Rehearsal – Class Inheritance

```
#include <iostream>
#include <string>
using namespace std;
class Vehicle {
public:
  string brand = "Ford";
  void honk() {
    cout << "Tuut, tuut! \n";
  }
};
class Car : public Vehicle {
public:
  string model = "Mustang";
};
int main()
{
Car myCar;
myCar.honk();
cout << myCar.brand + " " + myCar.model;
  return 0;
}
```

What's the output ?

Rehearsal – Class Inheritance

```
#include <iostream>
#include <string>
using namespace std;
class Vehicle {
public:
  string brand = "Ford";
  void honk() {
    cout << "Tuut, tuut! \n";
  }
};
class Car : public Vehicle {
public:
  string model = "Mustang";
};
```

Tuut, tuut!

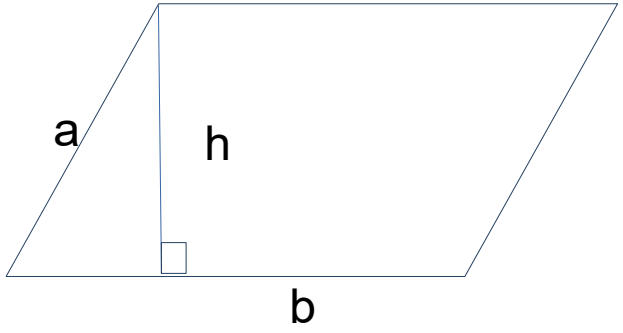
Ford Mustang

Exercise

Use **Shape2D** abstract class to derive a new **Parallelogram** class, similar to **Rectangle**

Implement Perimeter and Area calculations

- Perimeter = 2(a+b)
- Area = b * h (base*height)



Exercise

```
#include "Shape2D.h"
class Parallelogram : public Shape2D { // Derived class inheriting from Shape2D
private:
   double base;
   double side;
   double heigth;
public:
   // Constructor
   Parallelogram(double bas, double sid, double hei) : base(bas), side(sid), heigth(hei) {}
   // Override the virtual member function to calculate the area
   double calculateArea() const override {
      return base*heigth; // area of a parallelogram
   }
   // Override the virtual member function to calculate the perimeter
   double calculatePerimeter() const override {
      return 2*(side + base); // perimeter of a parallelogram
   }
};
```

Exercise – Japanese Demography

TRUE OR FALSE ?

*« **if** birth rate declines at today's pace, the last japanese child will be born in 2720 »* (Hiroshi Yoshida, japanese démograph)

Write some code to check that !

- Nb of births in 2024: **687000**
- Decrease compared to 2023: **2.3%**

Exercise – Compute prime Numbers

- Write code to print the prime number just **below** a given integer **N**
- Adapt it to print the first prime number **above N**

Software Quality

The 3 Biggest Software Lies:

- *« The program is fully tested and bug-free. »*
- *« We're working on the documentation. »*
- *« Of course we can modify it.»*

Software Reliability

- Ability of a system or component to perform its required functions under static conditions for a specific period.
- The probability that a software system fulfills its assigned task in a given environment for a predefined number of input cases, assuming hardware and inputs are free of error.
- An essential aspect of software quality, with functionality, usability, performance, maintainability, and documentation.
- Harder to achieve when software complexity increases.

Software Robustness

Ability of software to function correctly and reliably within unexpected, abnormal situations

- incorrect user input
- network failures
- hardware malfunctions
- deliberate attempts to disrupt the system.

Exception

« An unexpected problem that arises at runtime - during the execution of a program – causing it to terminate suddenly (crash) or behave in a no-longer predictible manner... »

Examples : Division by zero, Logarithm or square root of negative numbers, dereference a pointer, memory access violation by accessing beyond last index of an array...

Exception Examples

- Division by zero
- Logarithm of negative number
- square root of negative number
- dereference a pointer
- memory access violation by accessing an array beyond its last index…
- Stack overflow

Segmentation Fault

```
int* p = NULL;
*p = 1;
```

- There are many ways to get a segfault. A common way is to dereference a null pointer:

Stack Overflow

a type of buffer overflow error that occurs when a computer program tries to use more memory space in the call stack than has been allocated to that stack. The call stack, also referred to as the stack segment, is a fixed-sized buffer that stores local function variables and return address data during program execution.

Stack Overflow

```
int foo()
{
return foo();
}
```

- The function foo, when invoked, continues to invoke itself, allocating additional space on the stack each time, until the stack overflows, resulting in a segmentation fault.

« Dangling » Pointer

```
char* p = NULL;
{
char c;
p = &c;
}
// Now p is dangling
```

- The pointer p dangles because it points to the character variable c that ceased to exist after the block ended. And when you try to dereference dangling pointer (like *p='A'), you would probably get a segfault.

Software Reliability

```
char* p1 = NULL;
*p1 = 'a';
int* p2 = NULL;
{
int n;
p2 = &n;
}
int MyArr[10] = { -2, 2, -4, 4, -6, 6, -8, 0, 23, -33 };
*p2 = MyArr[5];
float y;
for (int i = 0; i <= 10; i++)
{
y = 1 / MyArr[i];
}
```

- What's wrong
  - in this code ?
- How many errors can you find?

Software Reliability

```
char* p1 = NULL;
*p1 = 'a'; // WRONG : dereference a null pointer
int* p2 = NULL;
{
int n;
p2 = &n;
}
int MyArr[10] = { -2, 2, -4, 4, -6, 6, -8, 0, 23, -33 };
*p2 = MyArr[5];
float y;
for (int i = 0; i <= 10; i++)
{
y = 1 / MyArr[i];
}
```

- What's wrong
- in this code ?

Software Reliability

```
char* p1 = NULL;
*p1 = 'a'; // WRONG : dereference a null pointer
int* p2 = NULL;
{
int n;
p2 = &n; // WRONG : address contained in p2 exists only within {}
}
int MyArr[10] = { -2, 2, -4, 4, -6, 6, -8, 0, 23, -33 };
*p2 = MyArr[5];
float y;
for (int i = 0; i <= 10; i++)
{
y = 1 / MyArr[i];
}
```

- What's wrong
- in this code ?

Software Reliability

```
char* p1 = NULL;
*p1 = 'a'; // WRONG : dereference a null pointer
int* p2 = NULL;
{
int n;
p2 = &n; // WRONG : address contained in p2 exists only within {}
}
int MyArr[10] = { -2, 2, -4, 4, -6, 6, -8, 0, 23, -33 };
*p2 = MyArr[5];// WRONG : p2 cannot be used, it is dangling
float y;
for (int i = 0; i <= 10; i++)
{
y = 1 / MyArr[i];
}
```

- What's wrong
- in this code ?

Software Reliability

```
char* p1 = NULL;
*p1 = 'a'; // WRONG : dereference a null pointer
int* p2 = NULL;
{
int n;
p2 = &n; // WRONG : address contained in p2 exists only within {}
}
int MyArr[10] = { -2, 2, -4, 4, -6, 6, -8, 0, 23, -33 };
*p2 = MyArr[5];// WRONG : p2 cannot be used, it is dangling
float y;
for (int i = 0; i <= 10; i++)// WRONG : loop will try to write on 11th element
{
y = 1 / MyArr[i];
}
```

- What's wrong
- in this code ?

Software Reliability

```
char* p1 = NULL;
*p1 = 'a'; // WRONG : dereference a null pointer
int* p2 = NULL;
{
int n;
p2 = &n; // WRONG : address contained in p2 exists only within {}
}
int MyArr[10] = { -2, 2, -4, 4, -6, 6, -8, 0, 23, -33 };
*p2 = MyArr[5];// WRONG : p2 cannot be used, it is dangling
float y;
for (int i = 0; i <= 10; i++)// WRONG : loop will try to write on 11th element
{
y = 1 / MyArr[i];// WRONG : division by zero for i = 7
}
```

- What's wrong
- in this code ?

Exception handling - C++ Try and Catch

3 keywords: **try**, **throw** and **catch**:

- **try** statement allows you to define a block of code to be tested for errors while it is being executed.
- **throw** keyword throws an exception when a problem is detected, which lets us create a custom error.
- **catch** statement allows you to define a block of code to be executed, if an error occurs in the try block.

C++ Try and Catch

The **try** and **catch** keywords come in pairs:

```
try {
// Block of code to try
throw exception; // Throw an exception when a problem arise
}
catch () {
// Block of code to handle errors
}
```

C++ Try and Catch - Example

```
try {
int age = 15;
if (age >= 18) {
cout << "Access granted - you are old enough.";
}
else
    throw (age);
}
catch (int myNum) {
cout << "Access denied - You must be at least 18 years old.\n";
cout << "Age is: " << myNum;
}
```

We use the **try** block to test some code: If the age variable is less than 18, we throw an exception, and handle it in our catch block.
In the **catch** block, we catch the error and do something about it. The catch statement takes a parameter: in our example we use an int variable (myNum) (because we are **throwing** an exception of int type in the try block (age)), to output the value of age.
If no error occurs (e.g. if age is 20 instead of 15, meaning it will be be greater than 18), the **catch** block is skipped:

C++ Try and Catch - Example

```
try {
int age = 15;
if (age >= 18) {
cout << "Access granted - you are old enough.";
}
else {
throw 505;
}
}
catch (int myNum) {
cout << "Access denied - You must be at least 18 years old.\
n";
cout << "Error number: " << myNum;
}
```

You can use the throw keyword to output a **reference number**, like an **error number/code** for organizing purposes (**505** in our example):

C++ Assert

- An assertion statement specifies a condition that you expect to be true at a point in your program. If that condition is not true, the assertion fails, execution of your program is interrupted, and the Assertion Failed dialog box appears
- It is normally bad practice to use asserts in deployed software, because it typically provides information that is useful only to programmers; exceptions are preferred in this case.

C++ Assert

- Assertions are statements used to test assumptions made by programmers. For example, we may use assertion to check if the pointer returned by malloc() is NULL or not.

```
void assert(int expression);
```

C++ Assert

```
#include <assert.h>
int main(){
int x = 7;
/* here x is set to 9 accidentally
x = 9;
// x assumed = 7 in rest of the code
assert(x == 7);
/* Rest of the code */
return 0;
}
```

Output :

Assertion failed: x==7, file test.cpp, line 13
This application has requested the Runtime to terminate it in an unusual way. Please contact the application's support team for more information.

Ignoring C++ Assertions

In C/C++, we can completely remove assertions at compile time using the **preprocessor directive** **NDEBUG**. It is set by default in the DEBUG Configuration

See example in Visual Studio Project Properties

Software Performance

« Think Performance EARLY and OFTEN»

Tom Gilb

Software Performance

- Is a « non functional requirement «
- Defined as « how efficientlly a software can accomplish its tasks ».
- As software can be very complex and be formed by a wide variety of components & parts
- Evaluating the performance of such systems might be quite complex.

Non-functional Requirements

- Define **how a system should behave**, rather than what it is supposed to do.
- Unlike functional requirements, which describe specific system functions, NFR define aspects like **performance**, security, usability, reliability, and scalability.

Software Performance - Scope

Questions to ask - base yourself on typical **Use Cases**

- Performance scope? Subsystems, interfaces, components
- If not within scope => out of scope for this requirement
- User interfaces (Uis) : nb of concurrent users ? (peak and nominal)
- Hardware specs (server, network appliances, CPU, memory...)?
- Application Workload for each component? (for example: 20% log-in, 40% search, 30% item select, 10% checkout).
- Time requirements (peak vs. nominal)?

Software Performance Criteria

Examples of Performance requirements :

- **Login time <** 10 seconds
- **Algorithm** (**FFT**) on acoustic waveforms : « should process a 10000 samples waveform within less than 100ms »
- **ATM Machine** : A basic transaction should be complete within 1 minute
- **Web Server** should support concurrent users up to 1 million without noticeable performance issue

Achieving Software Performance

A system is never more performatic than its slowest part and that part is what we call a **bottleneck.**

to improve the performance of a system, one has to improve the performance of its **slowest part**, as all your proccessing is queueing in there and all the rest of your system haven't reached its peak yet.

Assessing Software Performance

Use Profiling Tools

- Visual Studio Profiler

Achieving Software Performance

- Build Config : use Optimize option
- Spot Performance bottlenecks
- optimize calculations in big loops
- Avoid pointer Dereference in big loops
- Use float not double unless you really need it
- Use multiplication rather than pow()
- Etc..

Optimizing C++ Code

« **Round up the usual suspects** »

Capt. Louis Renault (Claude Rains), Casablanca movie, 1942

C++ optimization “usual suspects” :

- **function calls**
- **memory allocation**
- **loops**.

- Use a Better Compiler, Use Your Compiler Better
- Use Better Algorithms
- Use Better Libraries
- Reduce Memory Allocation and Copying
- Remove Computation from loops
- Use Better Data Structures
- Increase Concurrency
- Optimize Memory Management

Optimization Exercice

```
int arr[1000];
int a = 1, b = 5, c = 25, d = 7;
for (int i = 0; i < 1000; ++i) {
arr[i] = (((c % d) * a / b) % d) * i;
}
```

Can we optimize this code ?

Avoid calculations in big loops

```
int arr[1000];
int a = 1, b = 5, c = 25, d = 7;
// Calculating a constant
// expression for each iteration is
// not good.
for (int i = 0; i < 1000; ++i) {
arr[i] = (((c % d) * a / b) % d) * i;
}
```

```
int arr[1000];
int a = 1, b = 5, c = 25, d = 7;
// pre calculate the constant
expression
int temp = (((c % d) * a / b) % d);
for (int i = 0; i < 1000; ++i) {
arr[i] = temp * i;
}
```

Measuring your code execution time

```cpp
#include <chrono>
int main() {
using clock = std::chrono::system_clock;
using sec = std::chrono::duration<double>;
const auto before = clock::now();
int arr[100000];
int a = 1, b = 5, c = 25, d = 7;
// Calculate const expr for each iteration : not good.
for (int i = 0; i < 100000; ++i) {
arr[i] = (((c % d) * a / b) % d) * i;
}
const sec duration = clock::now() - before;
std::cout << " \n duration : " << duration.count() << "s";
return 0; // Return 0 to indicate successful completion
}
```

```cpp
#include <chrono>
int main() {
using clock = std::chrono::system_clock;
using sec = std::chrono::duration<double>;
const auto before = clock::now();
// pre calculating the constant expression outside loop
int temp = (((c % d) * a / b) % d);
for (int i = 0; i < 100000; ++i) {
arr[i] = temp * i;
}
const sec duration = clock::now() - before;
std::cout << " \n duration  : " << duration.count() << "s";
return 0; // Return 0 to indicate successful completion
}
```

Measuring Software Performance

- The **load** is the volume of transactions processed by the application, e.g., transactions per second, requests per second, pages per second. Without being loaded by computer-based demands (e.g. searches, calculations, transmissions), most applications are fast enough, which is why programmers may not catch performance problems during development.
- The **response** time is the time required for an application to respond to a user's action at such a load.
- **Computational** time : time required by the software to perform a complex calculation with a given set of input data (min, heavy, casual...)

Software Performance Load Testing

Intended to understand the behavior of the system under a specific expected load, such as :

- the expected concurrent **number of users** on the application performing a specific number of transactions within the set duration.
- This test will give out the response times of **all the important business critical transactions**. The database, application server, etc. are also monitored during the test, this will assist in **identifying bottlenecks** in the application software and the hardware.

Stress Testing

normally used to understand the upper limits of capacity within the system. This kind of test is done to determine the system's robustness in terms of extreme load and helps application administrators to determine if the system will perform sufficiently if the current load goes well above the expected maximum.

« Soak » (endurance) Testing

Serves to determine if the system can **sustain** the **continuous expected load**. During soak tests, memory utilization is monitored to detect potential **leaks**. Also important, but often overlooked is performance degradation, i.e. to ensure that the throughput and/or response times after some long period of sustained activity are as good as or better than at the beginning of the test. It essentially involves applying a significant load to a system for an extended, significant period of time.

Spike Testing

Done by suddenly increasing or decreasing the load generated by a very large number of users, and observing the behavior of the system. The goal is to determine whether performance will suffer, the system will fail, or it will be able to handle dramatic changes in load.

Exercise on Performance Optimization

```
const int nbloops = 1000000;
double func(){
using namespace std;
clock_t begin = clock();
//Code to measure here
double a = 2.45, b = 5.111, pi = 3.14;
for (int i = 0; i < nbloops; i++) { // loop
float x, y;
double c = (pow(a, 2) + pow(b, 2)) / pi;
x = sqrt(c);
y = pow(x, 2);} // end loop
clock_t end = clock();
double elapsed_secs = double(end - begin) .CLOCKS_PER_SEC;
return elapsed_secs;}
```

```
#include <iostream> // input/output
#include <cmath> // Include math
#include < ctime >
#include <chrono>
using namespace std;
int main() {
double elapsedsec = func();
std::cout << " \n duration : " <<
elapsedsec << "s";
return 0;
}
```

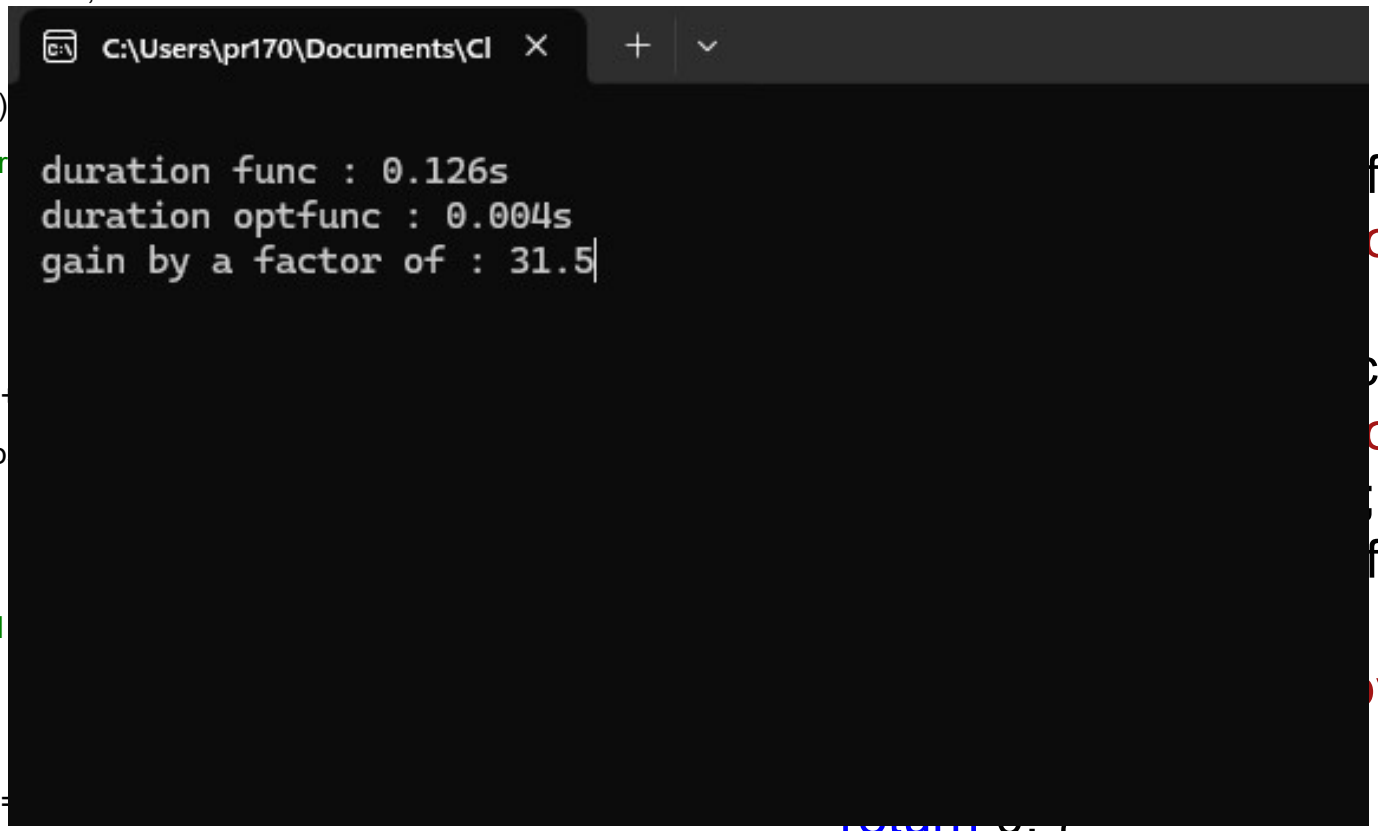
Solution

```
const int nbloops = 1000000;
double optfunc(){
clock_t begin = clock();
//Code to measure here
double a = 2.45;
double b = 5.111;
double pi = 3.14;
double c = (pow(a, 2) + pow(b, 2)) / pi;// calculation extracted from loop
for (int i = 0; i < nbloops; i++) { // loop
float x, y;
x = sqrt(c);
y = x*x;//pow replaced by multiplication
}
clock_t end = clock();
double elapsed_secs = double(end - begin) / CLOCKS_PER_SEC;
return elapsed_secs;
}
```

```
int main() {
double elapsedfunc = func();
std::cout << " \n duration func : " <<
elapsedfunc << "s";
double elapsedoptfunc = optfunc();
std::cout << " \n duration optfunc : " <<
elapsedoptfunc << "s";
double gain = elapsedfunc /
elapsedoptfunc;
std::cout << " \n gain by a factor of : "
<< gain;
return 0; }
```

Solution

```
const int nbloops = 1000000;
double optfunc(){
clock_t begin = clock()
//Code to measure her
double a = 2.45;
double b = 5.111;
double pi = 3.14;
double c = (pow(a, 2) +
for (int i = 0; i < nbloop
float x, y;
x = sqrt(c);
y = x*x;//pow replaced
}
clock_t end = clock();
double elapsed_secs =
return elapsed_secs;
}
```




```
func();
on func : " <<
= optfunc();
on optfunc : " <<
func /
y a factor of : "
```